

Handout 01

Routing Algorithm Benchmarks — Poland & Europe

Eduard Tykhoniuk

2026-06-13

Contents

1	Introduction	2
2	Experimental Setup	2
3	Used Algorithms	3
4	Poland Results	3
5	Random-Pair Queries	4
6	Europe Results	4
7	Discussion	4
8	Failures and Learning Process	5
	Result JSON Files Used	7

1 Introduction

The practical goal of the work was to build a small C++ routing-algorithm library and benchmark its one-to-one shortest-path algorithms on real-world road graphs imported from Geofabrik OpenStreetMap extracts. The code was generated by AI. My role was to review pull requests, check that the implementations matched the algorithms described in the handouts.

The main comparison uses the Poland graph, where all implemented algorithms fit within the intended memory limit. The initial intend was to compare algorithms on the Europe graph, however, due to the lack of RAM, only some of the algorithms were tested, more detailed in the Section 6.

All algorithms solve point-to-point shortest paths on directed weighted road graphs. The objective minimizes driving distance. The implementations compute only the shortest-path distance value; they do not reconstruct or output the actual sequence of vertices or edges on the path. For imported OSM edges, the edge length is computed from the two endpoint coordinates using Haversine distance, scaled by 1000, rounded up with `ceil`, and stored as `uint32_t`. A* uses the same coordinate model as an admissible lower bound but rounds down with `floor`. Integer arithmetic is faster than floating point and avoids floating-point nondeterminism.

2 Experimental Setup

- CPU: AMD Ryzen 7 5700X, 8 cores / 16 threads, max boost 4.9 GHz.
- RAM: 31 GiB physical, 99 GiB swap.
- Swap backing device: NVMe at 2.7 GB/s.
- OS: Linux 7.0.10-arch1-1.

The intended benchmark memory policy was a 24 GiB RAM cap for algorithm-owned working sets. Swap was avoided to prevent swap I/O from distorting benchmark timings.

Table 1: Graph statistics.

Graph	Vertices	Directed edges	File size	Source
Poland	17,058,998	33,674,745	675 MB	Geofabrik, extract 2026-05-27
Europe	370,316,072	731,016,367	14.7 GB	Geofabrik, extract 2026-05-27

For the Europe extract, the node map did not fit in RAM, so a custom parser was written that processes the data across multiple files and merges them into a single graph.

Query Methodology

Two query types were used.

Ranged-Dijkstra forms the main comparison. A Dijkstra run starts from a random source, settles up to $N_{\max}$ vertices, and samples the target from settled ranks $[N_{\min}, N_{\max}]$; this selects medium-distance routes and stores reference distances for validation.

- Query count: 1 000, seed 1
- Settled range: $N_{\min} = 10^5$, $N_{\max} = 10^6$

Random pairs use uniformly random source–target pairs with no distance constraint and no stored reference distance. They appear in a separate section.

- Query count: 1 000, seed 2

Each replay benchmark runs one algorithm per process. CH-dependent algorithms load the prebuilt Poland CH artifact where possible, so dependency load/build time is reported separately from algorithm-specific preprocessing.

3 Used Algorithms

The implemented algorithms used in the report are: Dijkstra, A*, ALT, Bidirectional Dijkstra, Bidirectional A*, Contraction Hierarchy (CH), Arc Flags, CHASE, and Hub Labels (HL).

A* and Bidirectional A* Haversine lower bound: $h(v, t) = \lfloor \text{haversine_m}(v, t) \times 1000 \rfloor$.

ALT Main benchmark uses 16 farthest-point landmarks with fixed seed and 4 active landmarks per query. Table 5 compares farthest, random, and planar selection strategies.

Arc Flags Main row uses 32 Inertial regions and 16 preprocessing threads. Flags are stored in a 64-bit region mask, so unsupported region counts above 64 are rejected rather than truncated.

CHASE Uses 64 Inertial regions and a 5% CH core fraction.

Hub Labels `label_fraction=0.05` due to high memory usage (≈ 24 GiB peak).

4 Poland Results

All main Poland rows use 1 000 ranged queries with seed 1. The generated tables below are produced directly from result JSON files, and the exact filenames are listed at the end of the report.

Preprocessing

Table 2: Preprocessing times and peak RSS — Poland graph.

Algorithm	Dep. wall (s)	Dep. CPU (s)	Algo wall (s)	Algo CPU (s)	RSS after	Notes
Dijkstra	<0.001	<0.001	<0.001	<0.001	844 MB	—
A*	<0.001	<0.001	<0.001	<0.001	1.1 GB	—
Bidirectional Dijkstra	<0.001	<0.001	1.0	0.9	1.8 GB	—
Bidirectional A*	<0.001	<0.001	1.0	0.9	2.0 GB	—
ALT (16 landmarks)	<0.001	<0.001	13.3	13.2	6.2 GB	—
CH	<0.001	<0.001	425.0	422.8	8.7 GB	CH saved, +64M aux. edges
Arc Flags (32 regions)	2.5	2.4	1 984.6	29 978.0	6.9 GB	CH loaded
CHASE (64 regions)	2.3	2.3	16.2	16.1	3.4 GB	CH loaded
Hub Labels (0.05)	2.3	2.2	397.4	394.5	24.0 GB	CH loaded

Query performance

Hub Labels is the fastest measured algorithm on Poland. After HL, CH is the fastest full-coverage CH-family baseline in this implementation. CHASE is close to CH but does not improve query time because CH already leaves a tiny search space for the extra flag checks to prune.

The unfortunate result for Arc Flags is that substantial preprocessing produces only a slight query speedup. Inertial partitioning is geometric rather than graph-aware, so many edges cross region boundaries and carry multiple region bits. The 1-to-16 thread preprocessing change mostly exposes memory-bandwidth contention over large PHAST and flag arrays.

Table 3: Query latency — Poland graph, 1 000 queries, seed 1, settled $\in [10^5, 10^6]$.

Algorithm	Mean (μ s)	P50	P95	P99	Mean CPU (μ s)	Speedup [†]	Validated
Dijkstra	54 379	53 304	99 190	107 870	54 102	1.0 $\times$	✓
A*	21 027	16 370	54 021	74 173	20 921	2.6 $\times$	✓
Bidirectional Dijkstra	44 100	38 290	104 369	136 532	43 871	1.2 $\times$	✓
Bidirectional A*	49 435	38 825	125 996	193 506	49 184	1.1 $\times$	✓
ALT (16 landmarks)	24 252	17 361	65 848	83 936	24 127	2.2 $\times$	✓
CH	169.0	158.7	318.9	392.7	168.4	322 $\times$	✓
Arc Flags (32 regions)	20 011	17 405	43 948	50 709	19 910	2.7 $\times$	✓
CHASE (64 regions)	179.9	164.7	341.3	423.0	179.2	302 $\times$	✓
Hub Labels (0.05)	78.5	78.4	111.7	132.2	78.1	693 $\times$	✓

[†]Speedup = Dijkstra mean wall / algorithm mean wall.

Table 4: Arc Flags configuration sweep — Poland graph, 1 000 ranged queries, seed 1.

Variant	Threads	Pre. wall	Pre. CPU	RSS	Mean (μ s)	P99 (μ s)	Prune rate	Speedup
Inertial-32	1	71.4 min	4 253 s	4.9 GB	21 668	57 648	2.7%	2.5 $\times$
Inertial-32	16	33.1 min	29 978 s	6.9 GB	20 011	50 709	2.7%	2.7 $\times$
Inertial-64	16	49.9 min	41 951 s	6.9 GB	14 997	50 176	4.3%	3.6 $\times$
KaMinPar-64	16	8.2 min	7 215 s	7.2 GB	12 410	29 331	4.5%	4.4 $\times$

Prune rate = mean pruned arcs / mean relaxed arcs per query.

5 Random-Pair Queries

All Poland algorithms were also benchmarked on 1 000 uniformly random source–target pairs (seed 2). Random pairs do not carry stored reference distances, so results are timing-only.

6 Europe Results

The Europe graph is large enough that memory behavior dominates the feasible benchmark set. Dijkstra, A*, and Bidirectional Dijkstra were run; ALT, CH, Arc Flags, CHASE, Hub Labels, and Bidirectional A* exceed the intended RAM cap.

Preprocessing

Query performance

A* works well at this big scale because the Haversine heuristic is a very good approximation for long geographic routes. It needs the graph, coordinate sidecar, and scratch arrays, but it does not need and preprocessing. In contrast, ALT would need roughly 47 GB for 16 forward/reverse landmark tables on 370 M vertices, and CH-family preprocessing is far beyond the available RAM.

7 Discussion

The main comparison is useful but not a final best-possible performance study: the algorithms were implemented functionally and with attention to obvious hot paths, but not exhaustively optimized. That caveat matters especially for Arc Flags and Hub Labels, where implementation details can strongly affect preprocessing memory and query latency.

On Poland, CH and Hub Labels dominate query latency because their preprocessing compresses the search space. A* is the best low-preprocessing option: it relies only on coordinates and

Table 5: ALT landmark-selection strategies — Poland graph, 1 000 ranged queries, seed 1.

Variant	Pre. wall (s)	RSS	Mean (μ s)	P99 (μ s)	Speedup
farthest-16-active-4	13.3	6.2 GB	24 252	83 936	2.2×
random-16-active-4	53.3	6.2 GB	8 601	47 667	6.3×
planar-16-active-4	24.6	6.2 GB	6 293	31 116	8.6×

Table 6: Random-pair check — Poland graph, 1 000 random pairs, seed 2.

Algorithm	Mean (μ s)	P50	P95	P99	Mean CPU (μ s)	Speedup [†]	Validated
Dijkstra	854 241	848 338	1 667 118	1 751 287	850 134	1.0×	—
A*	458 316	320 566	1 223 368	4 023 714	456 140	1.9×	—
Bidirectional Dijkstra	680 702	637 070	1 488 858	1 682 276	677 420	1.3×	—
Bidirectional A*	918 029	699 416	2 573 625	3 813 021	913 639	0.9×	—
ALT (farthest, 16/4)	456 801	310 669	1 266 264	2 716 578	454 591	1.9×	—
CH	695.5	744.8	1 104	1 209	692.2	1 228×	—
Arc Flags (KaMinPar-64)	31 489	28 266	66 801	108 520	31 327	27.1×	—
CHASE (64 regions)	797.0	852.6	1 317	1 485	792.6	1 072×	—
Hub Labels (0.05)	71.1	72.3	100.4	112.1	70.8	12 012×	—

Random pairs do not carry reference distances; `distance_validated=false`.

scratch memory. ALT improves over Dijkstra but pays a dense landmark-table cost. Arc Flags benefits from a graph-aware partition: [KaMinPar-64](#) (Table 4) improves over inertial partitioning.

CH class family of algorithms worked very well, as they greatly compressed the number of vertices in the optimal path. This also may be confirmed by the fact, that speedup is higher on completely random pairs as the search space does not rely on the underlying (real) shortest path’s length.

However, this family requires long preprocessing time as well as a high memory usage. On this hardware, none of the preprocessing algorithms worked for Europe. The other family of algorithms is low-preprocessing algorithms: A* was the best Europe-scale option.

8 Failures and Learning Process

CH and Arc Flags artifact saving was added because their preprocessing is the same between benchmark runs. Reusing those artifacts avoids rerunning the same expensive preprocessing step and makes it much faster to get benchmark results. Automated integration tests were also added to cross-check algorithm distances against Dijkstra reference results.

Memory pressure was the main practical limitation. Large benchmark batch sizes in PHAST consumed too much RAM, so the implementation was reduced to `kBatchSize = 1`. Hub Labels with `label_fraction=0.25` also turned out to be too large for the available memory, so the reported Poland run uses a smaller fraction.

For graph partitioning, [KaMinPar](#) was used as a newer graph-partitioning library than METIS. It improved over the simple inertial partitioning variant, but did not change the broader conclusion that Arc Flags remained preprocessing-heavy for the measured speedup.

Table 7: Preprocessing times and peak RSS — Europe graph.

Algorithm	Dep. wall (s)	Dep. CPU (s)	Algo wall (s)	Algo CPU (s)	RSS after	Notes
Dijkstra	< 0.001	< 0.001	< 0.001	< 0.001	17.8 GB	—
A*	< 0.001	< 0.001	< 0.001	< 0.001	23.3 GB	—
Bidirectional Dijkstra	< 0.001	< 0.001	116.9	95.9	28.4 GB	—

Table 8: Query latency — Europe graph, 1 000 queries, seed 1, settled $\in [10^5, 10^6]$.

Algorithm	Mean (μ s)	P50	P95	P99	Mean CPU (μ s)	Speedup [†]	Validated
Dijkstra	63 905	61 885	122 901	146 320	63 589	1.0×	✓
A*	25 129	20 098	63 040	93 098	25 001	2.5×	✓
Bidirectional Dijkstra	212 856	172 723	530 239	919 133	121 590	0.3×	✓

[†]Speedup = Dijkstra mean wall / algorithm mean wall.

Result JSON Files Used

Table group	JSON file	Commit	Variant	Graph	Query set	Queries
Main Poland	dijkstra_default.poland.20260613T040011.json		default	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	astar_default.poland.20260613T040113.json		default	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	bidijkstra_default.poland.20260613T040232.json		default	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	bidijkstra_default.poland.20260613T040317.json		default	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	alt_farthest-16-active-4.poland.20260613T203137.json		farthest-16-active-4	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	ch_default.poland.20260613T040417.json		default	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	arcflags_inertial-32_bs1_16t.poland.20260613T173814.json		32-partitions-inertial	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	chase_default.poland.20260613T042145.json		5pet-threshold-64-partitions-inertial	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Poland	hl_fraction-0.05.poland.20260613T113450.json		label_fraction=0.05, memory_budget=22GB	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Main Europe	dijkstra_default.europe.20260613T051741.json		default	europe-driving-20260613.graph	europe_ranged.q1000.s1.json	1000
Main Europe	astar_default.europe.20260613T190438.json		default	europe-driving-20260613.graph	europe_ranged.q1000.s1.json	1000
Main Europe	bidijkstra_default.europe.20260613T052957.json		default	europe-driving-20260613.graph	europe_ranged.q1000.s1.json	1000
Arc Flags sweep	arcflags_default.poland.20260613T041134.json		32-partitions-inertial	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Arc Flags sweep	arcflags_inertial-32_bs1_16t.poland.20260613T173814.json		32-partitions-inertial	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Arc Flags sweep	arcflags_inertial-64_bs1_16t.poland.20260613T181258.json		64-partitions-inertial	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Arc Flags sweep	arcflags_kaminpar-64_16t.poland.20260613T204500.json		64-partitions-kaminpar	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
ALT strategy	alt_farthest-16-active-4.poland.20260613T203137.json		farthest-16-active-4	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
ALT strategy	alt_random-16-active-4.poland.20260613T203227.json		random-16-active-4	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
ALT strategy	alt_planar-16-active-4.poland.20260613T203340.json		planar-16-active-4	poland-driving-20260613.graph	poland_ranged.q1000.s1.json	1000
Random pairs	dijkstra_random-pairs.poland.20260613T205100.json		random-pairs	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000
Random pairs	astar_random-pairs.poland.20260613T205100.json		random-pairs	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000
Random pairs	bidijkstra_random-pairs.poland.20260613T205100.json		random-pairs	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000
Random pairs	ch_random-pairs.poland.20260613T205100.json		random-pairs	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000
Random pairs	alt_random-pairs-farthest-16-active-4.poland.20260613T205100.json		random-pairs-farthest-16-active-4	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000
Random pairs	arcflags_random-pairs.poland.20260613T205100.json		random-pairs	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000
Random pairs	chase_random-pairs.poland.20260613T205100.json		random-pairs	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000
Random pairs	hl_random-pairs_fraction-0.05.poland.20260613T205100.json		random-pairs-label_fraction=0.05	poland-driving-20260613.graph	poland_random.q1000.s2.json	1000